ECS50-WI23-Lab1
ECS-50 Lab 1 ::
Basements and Byters

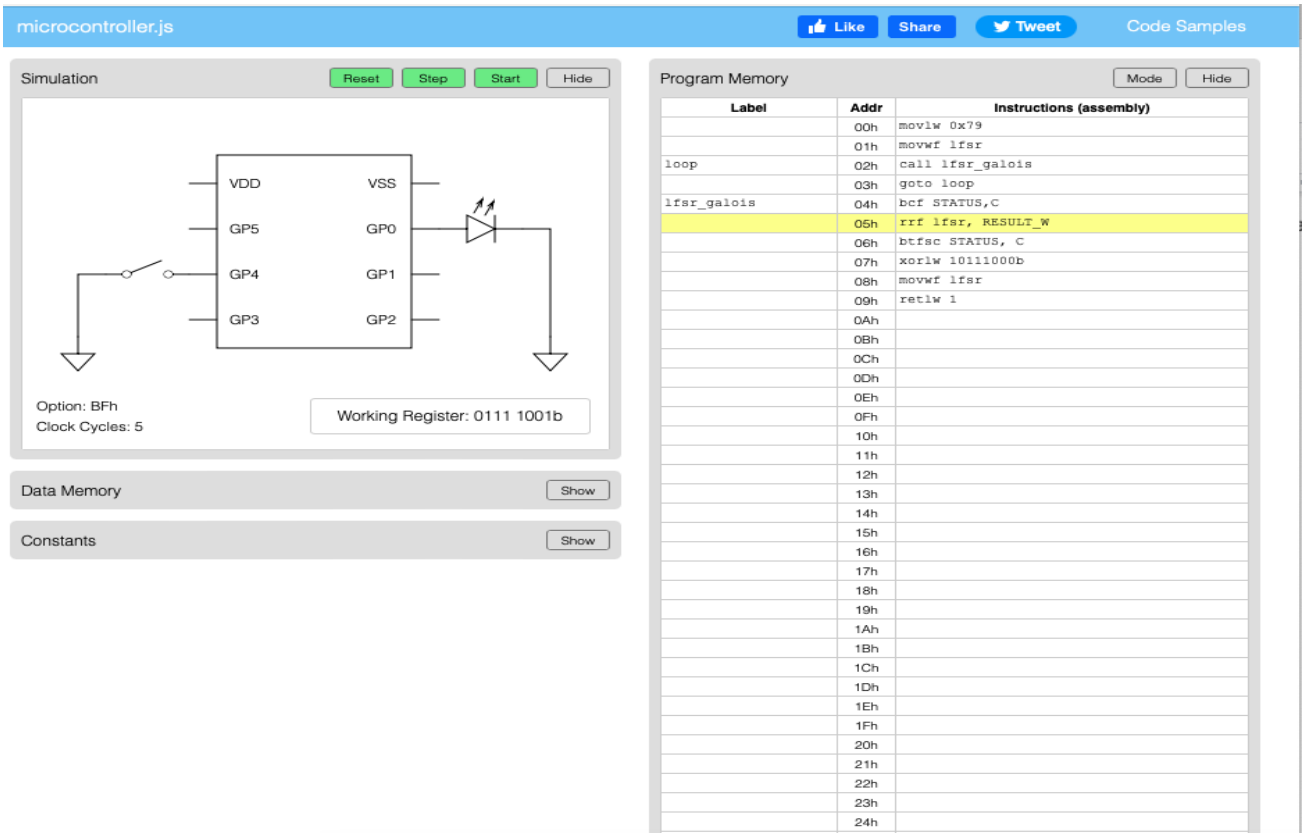
Sean Rhee

SID: 9820

Introduction:

In this lab, we are given two input ports that are represented by GP3 and GP4 which are both 4 binary bit ports. We are also given 4 output ports represented by GP0, GP1, GP2, and GP5. GP3 represents the ROLL switch and has the functions of "CLEAR" and "SET. Clear has the function of rolling the dice and Set displays the results as well as saving the results. GP4 determines what type of die will be rolled. In BnB, there are 2 different types of dice that we will use to play the game: 2d8z and 1d16z where d = the number of dice being rolled and z representing the values. 2d8z is used when the instruction is clear and the 1d16z die is used when the instruction is set. The goal of this lab was to determine the differences between 2d8z and 1d16z through the use of a pseudo number generator that was provided in the lab. The number generator is connected to the dice thrown in the game and depending on the number generated, it will determine which on/off switch is turned on.

Operation/Testing:



The first thing I did was take the pseudocode given in the lab to see how it is implemented in the microcontroller simulator. This serves as a way for the output to display numbers that will help our LED lights determine whether to turn on or off. The output values are represented in data memory and will be portrayed by the values. I will be using this code for my function *ifsr_galois.*

```
:origin

        movlw   00011000b
        tris    6
        clrf GPIO // this clears the field to start the game
```

I then began working on origin. The first thing I had to do was clear the field by using *clrf.*

```
:GP3_button // GP3 functions as the button needed to start the
roll
        btfss   GPIO, GP3
        btfsc   GPIO, GP3
        goto    GP3_button // this is a loop created

:GP4_rolls  // GP4 functions as the type of roll that will be
played
        btfss   GPIO, GP4
        movlw   TWO_ROLLS
        btfsc   GPIO, GP4
        movlw   ONE_ROLL
        movwf ROLL_COUNT
```

Next, I began creating functions for GP3 and GP4. As mentioned before, GP3 functions as a button that will start the roll and GP4 functions as the type of roll that will be played. I decided to use *btfss* because it bit-tests the functions and determines when it should move to the loop. Afterwards, I copied the code provided in the lab for the pseudo random binary number generator. The purpose of the number generator is for the values to be linked to the LED light display. All further details of the code behind the number generator was provided in the lab description.

```
:Button_Pressed //function that tells us whether to go to
single dice or two

        btfss GPIO, GP4
        btfsc GPIO, GP4
        goto single_dice

        call store_random_number
        btfsc ROLL_COUNT, 0
        goto lfsr_galois
        goto two_die


:store_random_number

        movf lfsr, w
        movwf store
        retlw 1


two_die

        movlw   00000111b
        andwf lfsr, f

        movlw   00000111b
        andwf store, f

        movf store, w
        addwf lfsr, f

        goto Display_LED

single_dice

        movlw   00001111b
        andwf   lfsr, f


Display_LED

        clrf GPIO
        btfsc   lfsr, 0
        bsf GPIO, GP0

        btfsc   lfsr, 1
        bsf     GPIO, GP1

        btfsc   lfsr, 2
        bsf     GPIO, GP2

        btfsc   lfsr, 3
        bsf     GPIO, GP5
```

Here I displayed the code for the functions when it calls for what type of dice to use as well as what to do when the dice is being used. Depending on the type of dice rolled, it will go to the *Display_LED* function. With the two dice function, I had it loop to the *Display_LED* in order for it to display the values that are connected to the dice. Each of the dice had a *mov* instruction

where I had the literal move to the w register. This then connects to the assembly code I made for *Display_LED*. *Display_LED* has the outputs GP0, GP1, GP2, and GP5 displayed as LEDs and using *btfsc,* we bit-test the values and allocate it to one of the ports so that it can fully display the LEDs.

```
@stack_ptr    0x0F
@stack_start    0x10
@stack_end    0x1F
$stack_size    0x10


:Circular_buffer //saves the last 16 rolls in locations 10h to
1fx.

// Derived from example 6
        movlw    stack_start
        addwf    stack_ptr, w
        movwf    FSR
        movf    lfsr, w
        movwf    INDF
        incf    stack_ptr, f
        movlw    00001111b
        andwf    stack_ptr, f
        goto GP3_button
```

We then made a circular buffer that saves the last 16 rolls in locations. This was derived from one of the examples in the simulator. Example 6 shows us how to implement a stack library. This is helpful to use because the buffer is very similar to a stack.

Conclusion

We were able to compile the code so that the inputs would output numbers that either turn on/off the LED lights. Using assembly code from the examples, we were able to produce a buffer for saving purposes as well as having GP3 and GP4 go through loops (*goto* instruction). A program was made to test what type of roll of dice will be played (single or double) and if the GP3 port is pressed down, then we can call a loop and check if it's set. If the input is set, we can

move onto the next instruction and determine what kind of dice/values will be displayed due to GP4.

Appendix (Copied From Simulator)

```
// Inputs and Outputs for BnB Game.
// GP0, GP1, GP2, GP5 represent outputs of values
// GP3, GP4 represent inputs
// Standard constants
$w      0
$f      1
$ONE_ROLL  1
$TWO_ROLLS      2
$RESULT_W  0
$RESULT_F   1

// Standard Variables
@ROLL_COUNT     0x0B
@store 0x0C
@lfsr 0x0A

:origin
        movlw 00011000b
        tris    6
```

```
        clrf GPIO // this clears the field to start the game
:GP3_button // GP3 functions as the button needed to start the roll
        btfss   GPIO, GP3
        btfsc   GPIO, GP3
        goto    GP3_button // this is a loop created
:GP4_rolls  // GP4 functions as the type of roll that will be played
        btfss   GPIO, GP4
        movlw           TWO_ROLLS
        btfsc   GPIO, GP4
        movlw           ONE_ROLL
        movwf ROLL_COUNT

//ECS 050 Code Copied from LAB

// Implements an 8-bit linear feedback shift register that can be used to generate a
// pseudo random binary number sequence. The output sequence can be viewed in
// the register lfsr.

// Copied ECS 050 Code
:rand_generator
        // Load seed value into lfsr
        movlw 0x79 // any non-zero value will work
        movwf lfsr
```

```
// imported from ECS 050 Code


:lfsr_galois
        // GP5 and GP1 flash on and off when rolling
        bsf     GPIO, GP5
        bsf     GPIO, GP1
        clrf GPIO

// First clear the carry so that we know that it is zero. Now, shift the lfsr right
// putting the LSB in carry and copying the lfsr into the working register for more processing

        bcf STATUS,C
        rrf lfsr, RESULT_W

// If the carry (LSB) is set we want to
// xor the lfsr with our xor taps 8,6,5,4 (0,2,3,4)

        btfsc STATUS, C
        xorlw 10111000b
        movwf lfsr // save the new lfsr
        btfss GPIO, GP3
```

```
        goto lfsr_galois
// end of ECS 050 Code Copied

:Button_Pressed

        btfss GPIO, GP4
        btfsc GPIO, GP4
        goto single_dice

        call store_random_number
        btfsc ROLL_COUNT, 0
        goto lfsr_galois
        goto two_die


:store_random_number

        movf lfsr, w
        movwf store
        retlw 1

:two_die
```

```
        movlw 00000111b
        andwf lfsr, f

        movlw 00000111b
        andwf store, f

        movf store, w
        addwf lfsr, f

        goto Display_LED

:single_dice

        movlw 00001111b
        andwf  lfsr, f


:Display_LED

        clrf GPIO
        btfsc    lfsr, 0
        bsf GPIO, GP0
```

```
        btfsc   lfsr, 1
        bsf     GPIO, GP1

        btfsc   lfsr, 2
        bsf     GPIO, GP2

        btfsc   lfsr, 3
        bsf     GPIO, GP5



@stack_ptr   0x0F
@stack_start   0x10
@stack_end   0x1F
$stack_size   0x10



:Circular_buffer //saves the last 16 rolls in locations 10h to 1fx.

// Derived from example 6
        movlw   stack_start
        addwf   stack_ptr, w
        movwf   FSR
        movf    lfsr, w
```

```
movwf    INDF
incf    stack_ptr, f
movlw    00001111b
andwf    stack_ptr, f
goto GP3_button
```